

LegalSAN: Compact Local Inference for Unfair-Clause Classification

Jiun-Yi Wu

Working paper · Revision 2 · 14 September 2026

Abstract

Can a bounded legal-language classification task be performed locally without a cloud inference service or a browser-managed generative model? We examine this question through LegalSAN, a task-specific encoder designed for application-controlled deployment. The model contains six attention-only blocks, a 12,000-entry embedding table, and 23,516,168 parameters including an eight-label classification head. Its ONNX graph integrates vocabulary-ID remapping, content masking, pooling, and classification. In a paired evaluation on 1,607 UNFAIR-ToS clauses, quantizing matrix operations and embedding lookups reduces the artifact from 94.35 MB to 24.04 MB, while eight-label macro-F1 changes from 0.7883 to 0.7870. Binary fair/unfair F1 is 0.8168, and 1,606 clause-level label vectors remain identical to the PyTorch reference. This revision adds an independent rerun of the released artifact, which reproduces the reference row of Table 1 exactly, and a nine-class evaluation on the 3,813-clause LegalBench unfair_tos split, where the same artifact reaches 95.88% accuracy and 88.46% binary macro-F1. Those two figures are reported against the majority baseline they must be read against: 90.58% of that split is the negative class, and the unfair class alone reaches F1 0.7902. These observations demonstrate a compact local classification path that does not invoke cloud inference or Gemini Nano. They establish task-specific feasibility rather than superiority over generative models. Browser-wide performance, multilingual generalization, and independently held-out evaluation of the final selection policy remain to be established.

Keywords: local inference; unfair-clause detection; compact encoders; model quantization; browser deployment

1 Introduction

Online terms present a practical information problem: users must make decisions about services whose contractual conditions may be difficult to inspect. Automated clause-level prompts can help identify passages that merit attention. For an assistant operating at the point of browsing, a central systems question is where the model should execute and who controls its deployment.

Cloud inference delegates execution to a remote service. Browser-managed AI provides another route: Chrome exposes APIs for models including Gemini Nano and manages the associated model lifecycle [17]. A task-specific model distributed with an application offers a third route. It can perform a fixed prediction task using an application-selected artifact and runtime, without depending on a cloud inference endpoint or a browser-provided generative-model API.

This study asks whether such an application-controlled classifier can retain useful benchmark performance within a model artifact budget of approximately 25 MB. The task is eight-label English unfair-clause classification, not open-ended legal reasoning. We investigate a compact encoder and measure how export and quantization affect storage, output scores, and thresholded decisions.

We make four contributions: (1) a self-contained inference graph for a 23.5-million-parameter clause classifier; (2) a paired comparison of two quantization scopes that isolates the storage benefit of including embedding lookups; (3) an analysis of decision agreement, category-level performance, and the remaining deployment requirements; and (4), new in this revision, an independent rerun of the released artifact together with a nine-class evaluation that reports the majority baseline alongside the headline figures. The evaluated pipeline demonstrates independence from cloud and browser-managed generative inference at the model level. It does not assume that all browsers or devices provide equivalent runtime performance.

The research contribution is the measured deployment case and its trade-offs. Established methods supply the architecture and compression ingredients. We therefore distinguish feasibility of a local alternative from a claim of better accuracy, speed, or generality than Gemini Nano or a cloud model, neither of which is directly evaluated here.

2 Related work and scope

BERT provides the pretrained encoder foundation [1], while LEGAL-BERT and the Contracts-BERT repository establish the legal-domain model lineage [2, 3]. UNFAIR-ToS is evaluated through LexGLUE [4]. The task itself predates this work: CLAUDETTE studied automatic detection of potentially unfair online terms [9]. Text to Trust evaluates BERT, DistilBERT, quantized adapter tuning, and prompting for unfair-term detection [13]. Its binary, rebalanced evaluation differs from the present eight-label test. Published scores under those protocols are not directly comparable to Table 1.

2.1 Attention-only encoders and architectural motivation

Our architectural motivation draws on A Controlled Study of Attention-Only Transformers [8], which examines the trade-off between feed-forward parameters and attention depth. That work studies decoder pretraining with matched-parameter, matched-compute, and matched-depth comparisons. LegalSAN instead uses a compact bidirectional encoder for supervised clause classification and examines its export. This extends the application context from decoder pretraining to task-specific encoder deployment; the effectiveness of the architectural choice is not isolated by the export experiment.

2.2 Vocabulary compression and combined pipelines

DistilBERT is an established example of encoder distillation [5]. Fast Vocabulary Transfer [10] combines domain-specific vocabulary adaptation with compression and evaluates LEDGAR, among other tasks. It learns a new tokenizer and transfers embeddings; the LegalSAN export retains original segmentation and remaps discarded IDs to an unknown embedding. Load What You Need [15] also reduces embedding vocabulary to obtain smaller multilingual encoders. Thus, identifying vocabulary storage as a compression target is established practice.

EI-BERT [12] combines hard token pruning, cross-distillation, and module-wise post-training quantization for edge NLU. It is a close precedent for combining these compression ingredients and reports a 1.91 MB model in different tasks and deployment settings. Our study focuses on six attention-only blocks and clause-level export, with a different evaluation objective. The reported model sizes do not provide a controlled performance comparison across tasks.

2.3 Quantization and local inference

Q8BERT [11] studies eight-bit BERT compression through quantization-aware fine-tuning. The present experiment instead uses ONNX Runtime dynamic post-training quantization [6] and compares two operator scopes. WebLLM [14] establishes browser-local language-model inference as an existing systems direction; ONNX Runtime Web [7] supplies the intended runtime here. Polisis [16] is an earlier neural system for privacy-policy analysis. These systems provide context for the present emphasis on a compact, application-controlled classifier for a fixed legal taxonomy.

2.4 Contribution boundary

This paper contributes a checkpoint-specific engineering evaluation: a custom encoder inference graph, a MatMul-versus-MatMul-plus-Gather quantization comparison, and paired numerical and label-decision checks on UNFAIR-ToS. The unit of analysis is one fixed checkpoint, `legalsan_finetuned.pt`, before and after export. This permits paired measurement of conversion effects while leaving comparisons between training recipes to future work. The full training provenance and evidence boundary are addressed in Section 7.

3 Model and inference graph

3.1 Compact checkpoint

Property	Recorded configuration
Encoder blocks	6; feed-forward sublayers absent
Hidden width / attention heads	768 / 12; 12 key-value groups
Vocabulary / positions	12,000 retained embeddings / 128 positions
Normalization	Post-attention LayerNorm; retained output LayerNorm
Task output	Mean pooling followed by a linear 8-label head
Total parameters	23,516,168, including classification head
Decision threshold	0.40 for every label

The encoder uses learned position and token-type embeddings. Each block applies multi-head self-attention with a residual connection and normalization, followed by the retained output normalization. It contains no feed-forward network. Evaluation mode disables dropout. Padded key positions receive an additive attention bias of $-10,000$ in the export implementation.

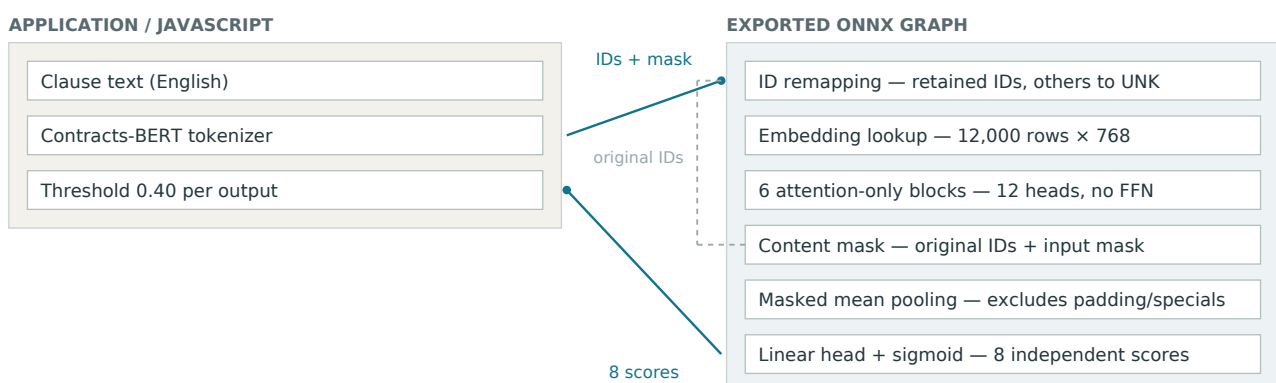
3.2 Token mapping and pooled representation

Tokenization uses the original 30,522-entry Contracts-BERT vocabulary. A lookup buffer maps its IDs to the retained 12,000-row table; dropped IDs map to the compact unknown-token ID. This preserves the original segmentation rather than re-tokenizing against a smaller vocabulary. It also means that discarded pieces lose lexical identity. The export study does not quantify coverage loss or performance by out-of-vocabulary rate.

The pooling mask excludes padding and original special-token IDs $\{0, 100, 101, 102, 103\}$. A dropped ordinary token remains in the pooling mask even when its embedding is remapped to the unknown row. For final hidden states h and binary content mask m , the graph computes:

$$z = (\sum_t m_t h_t) / \max(1, \sum_t m_t), p = \text{sigmoid}(Wz + b).$$

There is no L2 normalization of the pooled vector in this export path. Each label is predicted when its sigmoid output is at least 0.40. The eight outputs are independent scores, not a mutually exclusive distribution.



Runtime binaries and tokenization stay outside the model artifact.

Figure 1. The application tokenizes clause text; the ONNX graph handles remapping, embeddings, attention, content masking, pooling, and classification. The dashed branch supplies original IDs and the input mask to the content-mask operation. The input attention mask also masks padded keys inside attention (connection omitted for clarity). Runtime binaries and tokenization remain outside the model artifact.

4 Export and evaluation protocol

4.1 Export and quantization

Figure 1 summarizes the inference boundary. The wrapper accepts original token IDs and attention masks as INT64 tensors. ID remapping, content masking, pooling, the classification head, and sigmoid are embedded in the graph. Export uses ONNX opset 17 with dynamic batch and sequence axes. The learned position table still limits supported sequence length to 128 tokens. Structural validation uses the ONNX checker.

Two variants use ONNX Runtime dynamic quantization with signed INT8 weights, per-channel quantization enabled, and MatMulConstBOnly=True. The first targets MatMul; the second targets MatMul and Gather, thereby including eligible embedding lookups. Preprocessing succeeded in the recorded run. INT8 describes the quantization path, not a claim that every operation, activation, or graph value uses eight-bit arithmetic.

4.2 Data and metrics

The notebook loads UNFAIR-ToS from coastalcp/lex_glue, with 5,532 training, 2,275 validation, and 1,607 test examples. The export experiment evaluates the test split only. Whitespace is normalized; tokenization truncates and pads to 128 tokens. Inference uses batches of 32 and the CPU execution provider. The threshold of 0.40 is loaded from the checkpoint; its upstream selection history is not established by this notebook.

For each of eight labels, $F1 = 2TP / (2TP + FP + FN)$, with zero assigned for a zero denominator. Macro-F1 averages these eight scores. Micro-F1 aggregates counts over labels. Binary F1 collapses all labels into an any-unfair decision. Exact match compares the complete predicted vector with the ground truth; decision agreement instead compares each exported vector with the PyTorch vector. No ninth fair-class F1 is included in this protocol; Section 5.4 reports a separate nine-class protocol that does score the negative class.

The same checkpoint is evaluated before and after export, making drift comparisons paired. The selected quantization variant, however, is chosen using test macro-F1: the script falls back to MatMul-only if adding Gather loses more than 0.005. These are exploratory test results, not an untouched final assessment of a fully prespecified deployment policy.

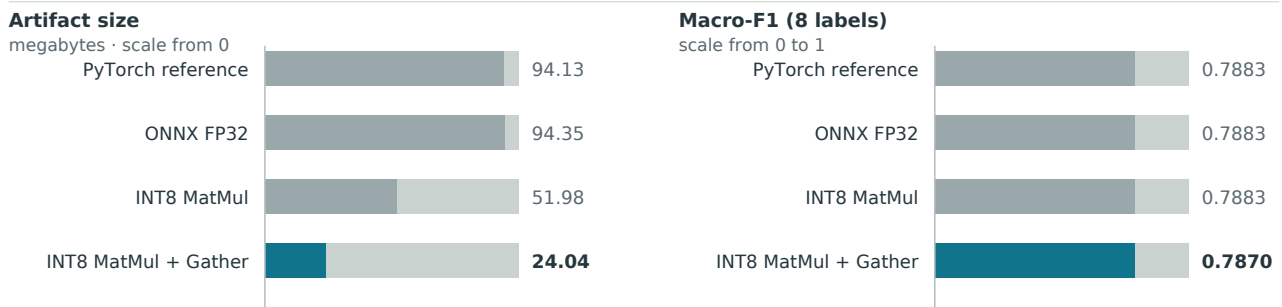
5 Results

Variant	MB	Macro-F1	Micro-F1	Binary F1	Exact match
PyTorch reference	94.13	0.7883	0.7619	0.8193	0.9527
ONNX FP32	94.35	0.7883	0.7619	0.8193	0.9527
INT8 MatMul	51.98	0.7883	0.7619	0.8193	0.9527
INT8 MatMul + Gather	24.04	0.7870	0.7598	0.8168	0.9521

Table 1. Recorded artifact sizes and test performance. MB denotes 10^6 bytes. Scores are rounded to four decimals.

Figure 2 visualizes the paired results. Relative to FP32 ONNX, MatMul-only quantization reduces size by 44.9%. Including Gather reduces size by 74.5%, or approximately 3.93 times, and saves a further 27.94 MB over MatMul-only. The $12,000 \times 768$ word-embedding table alone contains 9.216 million weights, or 36.864 MB in FP32. This explains why quantizing linear operations without embedding lookups leaves a substantial storage cost.

The notebook reports a macro-F1 change of -0.0012 from the unrounded values. Subtracting the independently rounded Table 1 endpoints gives -0.0013 ; this is a rounding difference. No confidence interval or multi-checkpoint quantization replication is available.



Full INT8 quantization is 74.5% smaller; macro-F1 moves by 0.0013.

Figure 2. Four versions of the same checkpoint, evaluated on the same 1,607 clauses at threshold 0.40. Size and macro-F1 are plotted as separate panels on their own zero-based scales, because a single pair of axes would imply a shared scale the two quantities do not have. Model size excludes runtime assets.

5.1 Numerical and decision consistency

Variant	Maximum absolute score difference	Whole-clause decision agreement
ONNX FP32	0.0000 (rounded)	100%
INT8 MatMul	0.0878	100%
INT8 MatMul + Gather	0.1821	99.94%

Table 2. Quantization drift across all 1,607 clauses, relative to the same PyTorch checkpoint.

The integrated wrapper matches the separate remap-and-pool implementation exactly on five example clauses. FP32 ONNX differs from the wrapper by at most 8.39×10^{-8} on those examples; a separate short-sequence check at length 16 yields 8.42×10^{-8} . These checks establish agreement for the tested inputs, not a proof over all possible sequences.

The rounded agreement of 0.9994 on 1,607 clauses corresponds to 1,606 identical label vectors. This does not specify how many individual labels changed within the remaining clause. The large maximum score difference also shows why decision stability at one threshold must not be interpreted as probability calibration or stability at arbitrary thresholds.

5.2 Category-level performance

Category	Precision	Recall	F1	Support
Limitation of liability	0.758	0.658	0.704	38
Unilateral termination	0.744	0.763	0.753	38
Unilateral change	0.750	0.632	0.686	38
Content removal	0.625	0.769	0.690	13
Contract by using	0.941	0.696	0.800	23
Choice of law	1.000	0.923	0.960	13
Jurisdiction	0.933	0.875	0.903	16
Arbitration	0.750	0.857	0.800	7

Table 3. INT8 MatMul + Gather performance. Support counts positive examples per label; multi-label supports need not sum to the number of clauses.

Seven categories achieve observed precision of at least 0.70. The notebook recommends withholding Content removal from the interface. Table 1 nevertheless evaluates all eight categories; it is not a measurement of a seven-category interface. Choosing categories from test precision is another

test-informed decision requiring subsequent independent evaluation.

Rare-category estimates are fragile. Arbitration has seven positive examples; its displayed precision and recall correspond to six true positives, two false positives, and one false negative. Choice of law has observed precision of 1.000, but only 13 positive examples. Neither observation warrants a broad reliability guarantee.

5.3 Independent rerun of the released artifact

Revision 1 transcribed its results from saved notebook outputs and stated that the checkpoint and exported binary had not been rerun during manuscript preparation. That gap is now closed. The deployed 24.04 MB artifact was rescored from scratch in the application repository, using the WordPiece implementation the browser worker ships rather than the notebook's tokenizer, at threshold 0.40. The procedure is released as `scripts/eval-unfair-tos.py`.

The rerun reproduces the Table 1 reference row to four decimals on all four metrics: macro-F1 0.7883, micro-F1 0.7619, binary F1 0.8193, exact match 0.9527. Seven of the eight rows of Table 3 reproduce exactly. The single exception is Unilateral change, where the rerun observes precision 0.774 and F1 0.696 against the recorded 0.750 and 0.686 — a difference of one false positive.

Two readings are consistent with this. Either the tokenizer reimplemention resolves one borderline clause differently, or the shipped artifact does not carry the single decision change that Table 2 attributes to Gather quantization. The two cannot be separated without the original binary, and the difference is one clause in 1,607 either way. What the rerun does establish is that the published aggregate figures describe the file the application actually serves, which is the property a deployment paper needs and the one revision 1 could not assert.

5.4 Nine-class evaluation with a scored negative class

The protocol of Section 4.2 scores eight unfair labels and never scores the fair case, so it cannot report an accuracy figure. LegalBench [18] redistributes the same underlying corpus as a single-label task over nine classes: the eight unfair categories plus an explicit Other for clauses that are not unfair. Its test split contains 3,813 clauses. We evaluate the same 24.04 MB artifact on it, predicting Other when no label reaches 0.40 and otherwise the highest-scoring label. The procedure is released as `scripts/eval-legalbench-unfair-tos.py`.

Class	Precision	Recall	F1	Support
Limitation of liability	0.787	0.643	0.708	98
Unilateral termination	0.779	0.757	0.768	70
Unilateral change	0.735	0.679	0.706	53
Content removal	0.667	0.640	0.653	25
Contract by using	0.756	0.816	0.785	38
Choice of law	0.958	0.821	0.885	28
Jurisdiction	0.903	0.875	0.889	32
Arbitration	0.733	0.733	0.733	15
Other	0.975	0.983	0.979	3,454

Table 4. Nine-class performance of the deployed INT8 artifact on the LegalBench unfair_tos test split, threshold 0.40. Unlike Table 3, the negative class is scored.

Overall accuracy is 95.88%, or 3,656 of 3,813 clauses. Collapsing the nine classes to fair against unfair gives a binary macro-F1 of 88.46%. Both figures are considerably higher than the eight-label macro-F1 of Table 1, and the reason is structural rather than substantive: Other is 3,454 of 3,813

clauses, so a classifier that labelled every clause fair would already score 90.58% accuracy. The measured accuracy stands 5.30 points above that baseline. Likewise, the binary macro-F1 averages an F1 of 0.9791 on the fair class against 0.7902 on the unfair class.

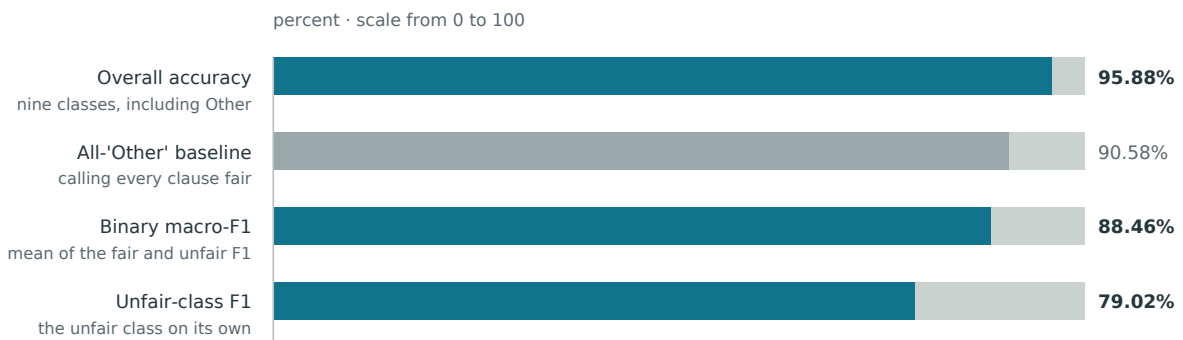


Figure 3. The two aggregate figures against the majority baseline and against the unfair class alone, on the same zero-based percentage scale. The fair majority lifts both aggregates; the unfair-class F1 is the quantity that describes finding unfair terms.

We therefore report 0.7902 — precision 0.822, recall 0.760, on 359 positive clauses — as the figure that characterises the system's actual task. It is the number a deployment decision should be based on, and an interface citing 95.88% without the 90.58% baseline beside it would overstate the model. Nine-class macro-F1, which weights Other equally with the rare categories, is 78.95%, close to the eight-label macro-F1 of Table 1 and a further indication that the accuracy figure's height comes from class balance.

5.5 Provenance of the nine-class split

The LegalBench split is not an independent sample. Matching clause text after whitespace and case normalization places every one of its 3,813 clauses somewhere in LexGLUE: 58.4% in the validation split, 40.4% in the test split, and 1.2% in the training split. A standard recipe would have used validation data for model or threshold selection, so a majority of the nine-class evaluation set is not held out in the sense the accuracy figure might suggest.

LexGLUE origin	Clauses	Share	Accuracy	Binary macro-F1
Training split	47	1.2%	97.87%	49.46%
Validation split	2,225	58.4%	95.78%	88.18%
Test split	1,541	40.4%	95.98%	88.98%

Table 5. The nine-class results decomposed by the LexGLUE split each clause originates from. The training-split row covers 47 clauses with one positive example and its binary figure is not interpretable.

The decomposition is reassuring rather than damaging. Performance on the 1,541 clauses that appear only in the LexGLUE test split — the portion no selection decision could have touched — is 95.98% accuracy and 88.98% binary macro-F1, slightly above the 95.82% and 88.10% measured on the remainder. Whatever advantage exposure might have conferred is not visible, and the headline figures are not inflated by the overlap.

This does not make the split held out. It means the two claims should be stated separately: the aggregate figures are reproducible on the released artifact, and they survive restriction to the never-selected-on subset. Establishing generalization to new documents still requires new documents, as Section 7.3 sets out.

6 Deployment boundary

The export emits an ONNX model, a 0.23 MB vocabulary, a configuration file, and a 0.08 MB tokenizer golden file. The latter contains 200 sampled test clauses plus six edge cases covering accents, punctuation, mixed alphanumeric text, whitespace, long words, and Chinese characters. These are reference token sequences for checking a JavaScript implementation. Generating them is not itself evidence that the JavaScript implementation passes.

6.1 Runtime evidence and remaining browser validation

The recorded native CPU timing is 7.4 ms per clause, averaged over 50 calls after five warm-up calls. It uses one fixed clause padded to 128 tokens and excludes tokenization and session initialization. CPU model and thread count are not recorded; the session uses default threading settings. This measurement is therefore a diagnostic observation, not a reproducible browser benchmark or a guaranteed lower bound on browser latency.

ONNX Runtime Web is the intended browser execution environment [7]. Preliminary project notes report a 49 ms WASM observation, tokenizer parity checks, and a live-page trial. Because the associated runtime harness and execution logs are not included in the experimental record, this paper bases its performance conclusions on the recorded native evaluation. A browser study should record device, browser and runtime versions, sequence lengths, warm-up, cold start, latency distribution, and peak memory.

Dynamic shapes support shorter inputs and variable batches, but larger batches are not assumed to be faster on every browser. The deployed package also includes runtime binaries and tokenizer assets, so 24.04 MB is the model artifact size, not the complete download or memory footprint. Long clauses require a documented segmentation or windowing policy rather than silent truncation.

6.2 Threshold as a released parameter

The decision threshold is part of the deployed system, not only of the evaluation. Every figure in this paper is measured at 0.40, and an application that flags clauses at a different value does not produce the performance the paper reports. During preparation of this revision the reference application was found to flag at 0.50 while advertising the Section 5.4 figures, which were measured at 0.40. The application now reads the threshold from the same constant the evaluation scripts use, and a unit test fails if the two diverge. The general point is that a threshold is a released artifact alongside the weights, and drift between them is silent.

7 Discussion and limitations

7.1 A task-specific alternative to generative inference

The central finding is that the evaluated classification operation can be performed by a compact encoder without calling a generative model. The 24.04 MB artifact produces eight task-specific scores locally. Its macro-F1 of 0.7870 and binary F1 of 0.8168 provide evidence that this is a substantive prediction path rather than only a demonstration of model loading. Practical usefulness still depends on the tolerated false-positive and false-negative rates in the intended interface.

The absence of cloud or Gemini Nano calls is a property of the implemented inference path; performance on UNFAIR-ToS supplies the complementary task evidence. This supports a claim of task-specific feasibility. It does not establish equivalence to a particular generative model, whose prompts, version, and output mapping would need to be controlled in a direct comparison. Training and artifact distribution are separate from inference: the present result does not imply that model development was cloud-free.

Application-controlled delivery also permits the developer to select and version the model artifact alongside the inference code. With required assets provisioned locally, classification need not transmit clause text to a remote inference service. The complete extension would still require

inspection of telemetry, logging, and network behavior before making an end-to-end privacy claim. Model download and runtime maintenance also remain operational costs.

7.2 Accuracy, uncertainty, and decision stability

Quantization preserves nearly all decisions at threshold 0.40, but the maximum absolute score change of 0.1821 shows that score stability is a stronger requirement. The evidence supports fixed-threshold classification more directly than confidence display or score-based ranking. Calibration, threshold sensitivity, and analysis of the changed clause are therefore important follow-up experiments.

Macro-F1, binary F1, exact match, and nine-class accuracy answer different questions, and the spread between them on one artifact is wide: 0.7870, 0.8168, 0.9521, and 0.9588 all describe the same model. The exact-match value measures full-vector agreement with ground truth and should not be substituted for the ability to detect rare unfair categories. The nine-class accuracy is bounded below by a 90.58% majority baseline and should never be quoted without it. Category-level results in Table 3 and Table 4 make the precision-recall trade-off explicit. Small positive supports preclude broad reliability claims for rare labels.

7.3 Threats to validity and next experiments

The export study uses a single checkpoint and one English benchmark. Quantization scope and the recommended displayed categories were selected using test results, so the deployed policy requires assessment on new, independently held-out documents. The nine-class evaluation of Section 5.4 does not supply that assessment: as Section 5.5 shows, its split is drawn from the same corpus, with a majority originating in LexGLUE validation data. Full upstream fine-tuning records, model-selection history, and repeated-seed measurements are not available in the experimental record used here. The paired export comparison remains informative about this checkpoint, but it does not estimate variability across trained models.

Native CPU execution verifies the graph on the measured platform. It does not establish browser-wide latency, memory use, or tokenizer parity. A full evaluation should measure cold start, steady-state latency distributions, peak memory, tokenization, and document processing on specified devices and browsers, including short sequences and variable batches. The 128-token limit requires explicit treatment of long clauses.

German AGB, cross-clause reasoning, and open-ended legal explanation are outside the present task. Classification indicates similarity to benchmark annotations, not a legal determination. Future experiments should compare equally constrained compact encoders, independently evaluate final display rules, and then test multilingual and real-site generalization. A generative-model baseline would answer a separate question about relative quality and cost.

8 Conclusion

LegalSAN demonstrates a compact local inference path for English unfair-clause classification without invoking cloud inference or a browser-managed generative model such as Gemini Nano. Joint MatMul and Gather quantization yields a 24.04 MB artifact with eight-label macro-F1 of 0.7870 and binary F1 of 0.8168, preserving the PyTorch label vector for 1,606 of 1,607 test clauses. An independent rerun of the released artifact reproduces the reference figures, and a nine-class evaluation on 3,813 clauses reaches 95.88% accuracy against a 90.58% majority baseline, with unfair-class F1 of 0.7902.

The contribution is evidence that a bounded consumer-facing language task can be implemented with an application-controlled encoder, together with a reproducible account of what its published numbers do and do not mean. Independent policy evaluation on new documents and complete browser measurements are the next steps toward establishing practical deployment suitability.

Data, code, and reproducibility

The primary experimental record is `export-onnx-browser.ipynb`: checkpoint configuration and parameter count (cells 3-4), wrapper checks (cell 6), export checks (cell 8), quantization sizes (cell 10), full-split results (cell 12), and per-label metrics and timing (cell 13). Cell indices are zero-based. Sections 1 through 5.2 are transcribed from saved outputs of that notebook.

Sections 5.3 through 5.5 were produced by rerunning the released artifact. The two evaluation scripts, `scripts/eval-unfair-tos.py` and `scripts/eval-legalbench-unfair-tos.py`, score `public/models/legalsan/legalsan-int8.onnx` — the exact file the browser downloads — through the tokenizer in `public/demo/legalsan-worker.js`, and print the tables reproduced here. Both default to threshold 0.40 and take the evaluation splits from their published sources at run time. This paper is built from `paper/legalsan-working-paper.md` by `paper/build.py`.

The recorded runtime for the original experiment is ONNX Runtime 1.29.0; installation requests Transformers 4.57.6 and version ranges for ONNX, datasets, and other dependencies. The export notebook does not pin dataset or tokenizer revisions, provide a complete resolved environment, or record checkpoint hashes. A reproducible release should still supply those identifiers, the upstream fine-tuning recipe, checkpoint-selection history, and per-example predictions. The model binary and the scoring path are now inspectable; end-to-end training reproducibility is not yet claimed.

Source notebook SHA-256: 2b53ef2fe037b175b6fd2f61d6eee8a64bc69cc4ba16b693cfd7e7861ba0b47c

Revision history

Revision 1 · 14 September 2026. Initial working paper, covering Sections 1 through 4, 5 through 5.2, 6.1, and 7.

Revision 2 · 14 September 2026. Adds Section 5.3 (independent rerun of the released artifact), Section 5.4 (nine-class evaluation with a scored negative class), Section 5.5 (provenance of that split), Section 6.2 (threshold as a released parameter), and Figure 3. Revises the abstract, Section 1, Section 4.2, Section 7.2, Section 7.3, Section 8, and the reproducibility statement to match. Figure 2 is redrawn as two panels rather than one pair of axes carrying two different quantities. Tables 1 through 3 and all prose from revision 1 are otherwise unchanged. The measurements added in this revision were produced by rerunning the released artifact with the scripts named above; they were not part of the original notebook record.

AI assistance. OpenAI ChatGPT assisted with drafting, code inspection, and document preparation for revision 1. Claude Code performed the reruns reported in Sections 5.3 through 5.5 and drafted those sections and this revision's edits. Neither executed new training or browser experiments.

References

- [1] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT, pp. 4171-4186.
- [2] Ilias Chalkidis et al. 2020. LEGAL-BERT: The Muppets straight out of Law School. Findings of EMNLP.
- [3] NLP AUEB. bert-base-uncased-contracts. Model repository and model card. Accessed 10 September 2026.
- [4] Ilias Chalkidis et al. 2022. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. ACL, pp. 4310-4330.
- [5] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108.
- [6] ONNX Runtime contributors. Quantize ONNX models. Documentation. Accessed 10 September 2026.
- [7] ONNX Runtime contributors. Web. Documentation. Accessed 10 September 2026.
- [8] Henry Ndubuaku et al. 2026. A Controlled Study of Attention-Only Transformers. arXiv:2607.18363.
- [9] Marco Lippi et al. 2018. CLAUDETTE: an Automated Detector of Potentially Unfair Clauses in Online Terms of Service. arXiv:1805.01217.
- [10] Leonidas Gee, Andrea Zugarini, Leonardo Rigutini, and Paolo Torroni. 2022. Fast Vocabulary Transfer for Language Model Compression. EMNLP Industry Track, pp. 409-416.

- [11] Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. Q8BERT: Quantized 8Bit BERT. arXiv:1910.06188.
- [12] Maolin Wang et al. 2025. Put Teacher in Student's Shoes: Cross-Distillation for Ultra-compact Model Compression Framework. KDD 2025. doi:10.1145/3711896.3737257.
- [13] Noshitha Padma Pratyusha Juttu et al. 2025. Text to Trust: Evaluating Fine-Tuning and LoRA Trade-offs in Language Models for Unfair Terms of Service Detection. arXiv:2510.22531.
- [14] Charlie F. Ruan et al. 2024. WebLLM: A High-Performance In-Browser LLM Inference Engine. arXiv:2412.15803; revised 2026.
- [15] Amine Abdaoui, Camille Pradel, and Gregoire Sigel. 2020. Load What You Need: Smaller Versions of Multilingual BERT. SustainLP, pp. 119-123.
- [16] Hamza Harkous et al. 2018. Polisis: Automated Analysis and Presentation of Privacy Policies Using Deep Learning. USENIX Security 2018.
- [17] Google. Built-in AI. Chrome for Developers. Accessed 14 September 2026.
- [18] Neel Guha et al. 2023. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. NeurIPS Datasets and Benchmarks. arXiv:2308.11462.